

Object Oriented Programming-II | Introduction Of OOP

Object-Oriented Programming (OOP) is a programming approach that organizes a program using classes and objects. It combines data and methods together into an object, making programs easy to develop, manage, and scale.

1. Main Features of OOP

- **Class** – A blueprint or template for creating objects.
- **Object** – An instance of a class containing state and behavior.
- **Encapsulation** – Binding data and methods together into a single unit while restricting direct external access.
- **Inheritance** – Acquiring properties and methods from an existing class to enable code reusability.
- **Polymorphism** – Allowing a single interface or method to perform different behaviors based on context.
- **Abstraction** – Hiding complex internal implementation details and exposing only essential features.

2. Difference Between SPL/SPM and OOP

Feature / Aspect	Structured Programming Language (SPL/SPM)	Object-Oriented Programming (OOP)
Primary Focus	Focuses on functions and procedures.	Focuses on objects and classes.
Data & Methods	Data and functions are generally separate.	Data and methods are combined inside objects.
Core Support	Does not natively support inheritance & polymorphism.	Fully supports inheritance, polymorphism, encapsulation, etc.
Data Security	Lower data security (unrestricted global data access).	Higher data security through encapsulation.
Code Reusability	Limited code reusability.	High code reusability via inheritance and modular classes.
Program Scope	Suitable for small and simple programs.	Ideal for large, complex, and scalable software systems.
Examples	C, Pascal, Fortran	Java, C++, C#, Python, PHP

3. Structured / Procedure Programming Methodology (SPM / SPL)

Structured Programming Methodology (SPM) or Structured Programming Language (SPL) is an approach where a program is divided into step-by-step functions or procedures. It mainly focuses on algorithms to solve a problem.

- **Examples:** C, Pascal, Fortran.

Basic Features of SPM/SPL

- **Algorithm-Focused:** Heavy emphasis is placed on step-by-step algorithms.
- **Function Decomposition:** Large programs are divided into smaller parts called functions.
- **Global Data Sharing:** Most functions share and manipulate global data.

- **Data Flow:** Data moves globally from one function to another.
- **Data Transformation:** Functions transform data from one form to another.
- **Top-Down Approach:** Programs follow a top-down execution design.

Limitations of SPM

- Reaches design limits when software projects become large and complex.
- Monolithic programs become complex, rigid, and hard to manage.
- Functions have unrestricted access to global data, leading to low data security.
- Code reusability is limited compared to OOP.
- Making changes in one part of a program may unexpectedly affect other parts.
- Lacks modern software design principles like inheritance, polymorphism, and encapsulation.
- Maintaining and debugging large codebases is difficult and error-prone.

4. OOP History — Important Points

- The concept of OOP emerged in the **1970s**.
- **Smalltalk** was the first purely object-oriented programming language, developed by **Alan Kay**.
- Popular OOP languages include **Java, C++, C#, Python, and PHP**.
- Examples of pure OOP languages include **Ruby, Scala, and JADE**.
- Languages primarily designed around OOP principles include **Java, Python, and C++**.

5. Importance & Benefits of OOP

- **Code Reusability:** Existing code can be reused easily through class inheritance.
- **Data Security:** Encapsulation protects sensitive object data from unauthorized external modification.
- **Easy Maintenance:** Modular design makes software easier to modify, refactor, and maintain over time.
- **Modularity:** Complex systems are broken down into independent classes and objects.
- **Flexibility:** Polymorphism allows a uniform interface to handle different underlying data types.
- **Easy Debugging:** Isolated objects make identifying and fixing bugs faster and simpler.
- **Reduces Complexity:** Abstraction simplifies software design by hiding engine-level mechanics.
- **Scalability:** Highly effective for enterprise-level, large-scale application development.

6. Advantages of OOP over SPL

1. **Code Reusability:** OOP reduces boilerplate code via inheritance and class templates.
2. **Data Security:** Protects data members using access modifiers (`private` , `protected`).
3. **Maintainability:** Clear separation of concerns makes troubleshooting and updates seamless.
4. **Modularity:** Encourages clean, object-driven domain design.
5. **Flexibility:** Dynamic method dispatch and polymorphism allow system capabilities to evolve easily.

Summary: OOP is far better suited for developing large, complex, and maintainable software than SPL.

7. Why Java is Not a Pure OOP Language

Java is considered an **Object-Oriented Programming (OOP)** language, but **not a pure OOP language** because it supports primitive data types that are not objects.

Main Reason: Primitive Data Types

Java supports **8 primitive data types**:

- `byte`, `short`, `int`, `long`
- `float`, `double`
- `char`, `boolean`

```
int x = 10; // 'x' is a primitive value, not an object reference
```

In a pure OOP language (such as Smalltalk or Ruby), *everything*—including numbers and boolean values—must be an object.

Additional Reasons:

1. **Static Members:** Static methods and variables can be accessed directly via class names without instantiating an object (e.g., `Math.sqrt(25);`).
2. **Primitive Operations:** Primitive variables operate directly on memory values rather than handling object message calls.
3. **Wrapper Classes:** Java provides wrapper classes (e.g., `Integer x = 10;`) to convert primitives to objects, which demonstrates that primitive types themselves are not native objects.

Conclusion: Since a pure OOP language mandates that every entity must be an object, Java is object-oriented, but not purely object-oriented.

8. Object-Oriented Analysis (OOA)

Object-Oriented Analysis (OOA) is the process of examining software requirements to identify real-world objects, classes, attributes, and relationships, creating a structural domain model before coding begins.

9. Core Concepts of OOP

The foundational elements of the object-oriented paradigm include:

A. Object

An **Object** is a real-world entity or an instance of a class that possesses **state** (data/attributes) and **behavior** (methods/actions).

```
Student s1 = new Student(); // s1 is an instance of the Student class
```

- **State:** Name, ID, Age
- **Behavior:** `study()`, `attendClass()`, `takeExam()`

B. Class

A **Class** is a blueprint or template used to define the state attributes and behavioral methods for objects created from it.

```
class Student {
    String name;
    int id;

    void study() {
        System.out.println("Student is studying");
    }
}
```

C. Methods

A **Method** is a block of code defined inside a class that represents the behavior or action of an object.

```
class Student {
    void study() {
        System.out.println("Student is studying");
    }
}
```

D. Instance Variables

Instance Variables are variables defined inside a class (outside methods) that belong to a specific object instance. Every object maintains its own independent copy of these variables.

```
class Box {
    double height;
    double width;
    double depth;
}
```

10. Unified Modeling Language (UML)

UML (Unified Modeling Language) is a standard visual modeling language used to specify, visualize, design, and document structural and behavioral components of software systems.

- **Visual Diagrams:** Uses standard diagrams (Class, Sequence, State diagrams) to model software architecture.

- **Blueprint for Software:** Enables architects and developers to design system domain models prior to coding.
- **OOAD Standard:** Serves as the global modeling standard in Object-Oriented Analysis and Design.

11. Four Basic Features (Pillars) of OOP

1. Abstraction

Abstraction represents essential features of an object while hiding non-essential internal implementation details.

- **Real-World Analogy:** A car driver uses the steering wheel, accelerator, and brakes without needing to understand internal engine combustion or gear mechanics.

2. Encapsulation

Encapsulation packages data and methods into a single class unit and hides internal state details from direct external access.

```
class Student {  
    private int age; // Encapsulated variable  
  
    public void setAge(int age) {  
        if (age > 0) {  
            this.age = age;  
        }  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

3. Inheritance

Inheritance allows a new class (subclass/child) to acquire the attributes and methods of an existing class (superclass/parent), enabling code reuse.

```

class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

```

- **Key Benefits:** Promotes code reusability, minimizes redundant code, and simplifies class extensions.

4. Polymorphism

Polymorphism allows an object or method to take multiple forms, performing different actions depending on context.

```

class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

```

Types of Polymorphism:

1. **Compile-time Polymorphism (Static Binding):** Implemented via **Method Overloading** (same method name with different parameter signatures).
2. **Run-time Polymorphism (Dynamic Binding):** Implemented via **Method Overriding** (subclass provides a specific implementation of a parent class method).